

Task Adaptive Parameter Sharing for Multi-Task Learning

2022 CVPR

Matthew et al.

Presenter

이재윤

Fundamental Team

이근배, 이세현

Contents

1. Introduction

2. Related Works

3. TAPS

4. Experiments

5. Conclusion

1. Introduction

Introduction

- Training, inference, and memory cost grows quickly with the number of tasks.
- Introducing single model to solve all tasks has several issues,
 - ✓ Deciding shared parameters
 - ✓ Performance degradation
 - ✓ etc.
- This paper introduce Task Adaptive Parameter Sharing (TAPS).
 - ✓ No additional parameters
 - ✓ No modifying base architecture
 - ✓ Adaptively selects a minimal subset of existing layers and retrains them
- Main things to see
 - ✓ How to select the best subset of layers
 - ✓ How to retrain those layers

2. Related works

Related Works

- Multi-Domain Learning (MDL)
 - ✓ Learn the domains incrementally (not all data is available at once).
 - ✓ Adapting to multiple domains incrementally poses the challenge of catastrophically forgetting past works
- Multi-Task Learning (MTL)
 - ✓ Learn diverse set of tasks in the same visual domain simultaneously
- Incremental Learning
 - ✓ Incrementally learn more classes
 - ✓ Catastrophic forgetting issue

Question

3. TAPS

Approach

- Given a pre-trained deep neural network with L layers of weights and a set of K target tasks $T = \{T_1, T_2, \dots, T_K\}$
- For each task, minimal subset of layers needs to be tuned
- In principle, this requires a combinatorial search over 2^L possible subsets
- TAPS relax the combinatorial problem

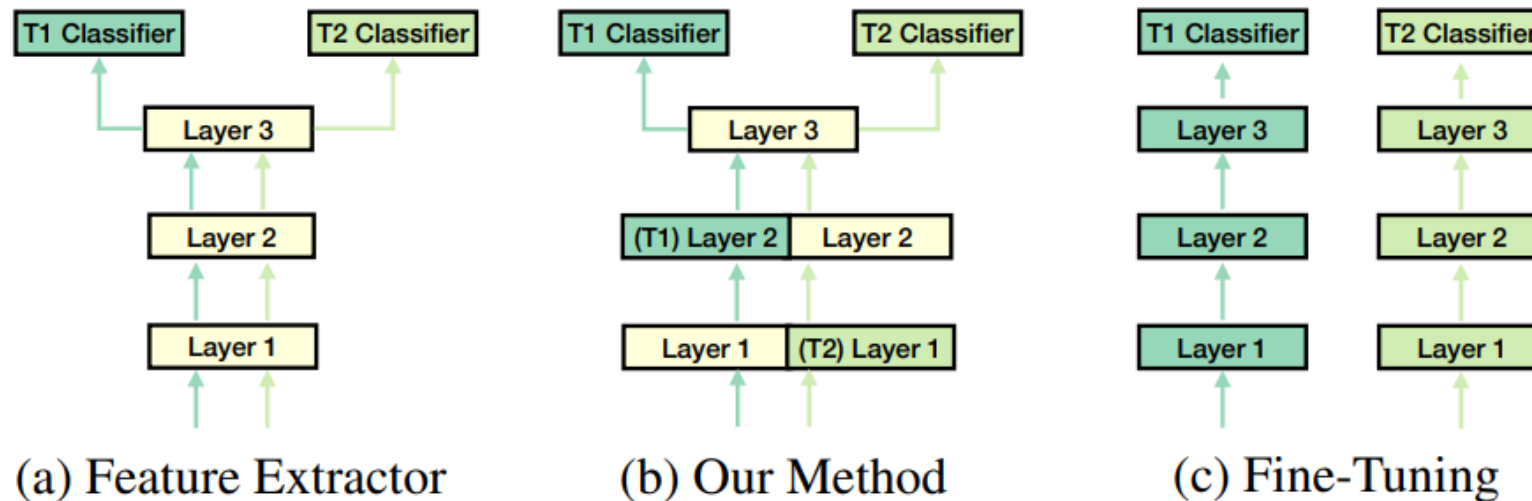


Figure 1. Overview of approach

Approach

- Weight Parametrization

$$w_i = \bar{w}_i + I_\tau(s_i) \delta w_i, \quad I_\tau(s_i) = \begin{cases} 1 & \text{if } s_i \geq \tau, \\ 0 & \text{otherwise.} \end{cases}$$

- ✓ Crucial component is indicator function I_τ
- ✓ If the score is bigger than threshold, the layer transformed to a task-specific layer
- ✓ If not, the layer remains as the base network
- ✓ Linear, convolutional, and attention

- w_i : weight of i - th layer
- $\bar{w}_i$: shared parameter of i - th layer
- I_τ : indicator function
- δw_i : task - specific perturbation
- s_i : scoring parameter
- τ : threshold

- Joint Optimization

$$(\delta \mathbf{w}, \mathbf{s}) = \arg \min_{\mathbf{w}, \mathbf{s}} \mathcal{L}_D(\mathbf{w}) + \frac{\lambda}{L} \sum_{i=1}^L |s_i|,$$

- ✓ First term optimize the task specific parameters to achieve the best performance on the task
- ✓ Second term is a sparsity which penalize large values of score s .
- ✓ Encourage layer sharing

- L : the number of layers
- λ : hyperparameter

Approach

- Straight-through gradient estimation

$$(\delta \mathbf{w}, \mathbf{s}) = \arg \min_{\mathbf{w}, \mathbf{s}} \mathcal{L}_D(\mathbf{w}) + \frac{\lambda}{L} \sum_{i=1}^L |s_i|,$$

- ✓ Equation above cannot be optimized with stochastic gradient descent
- ✓ The gradient of the indicator function is zero at all points (actually at point τ , gradient cannot be calculated)
- ✓ To make the problem learnable, modify the backward pass and redefine the gradient update as:

$$\nabla_{s_i} w_i = \delta w_i,$$

- Joint MTL

- ✓ In case of multi-task learning, data from multiple tasks is available simultaneously at training time

$$(\bar{\mathbf{w}}, \delta \mathbf{w}_1, \dots, \delta \mathbf{w}_K, \mathbf{s}_1, \dots, \mathbf{s}_K) = \arg \min_{\bar{\mathbf{w}}, \delta \mathbf{w}^k, \mathbf{s}} \sum_{k=1}^K \left(\mathcal{L}_{D_k}(\bar{\mathbf{w}}, \delta \mathbf{w}^k) + \frac{\lambda}{L} \sum_{i=1}^L |s_i^k| \right),$$

– K : the number of tasks

- Limitation

- ✓ Memory footprint during training increases linearly with the number of tasks

Question

4. Experiments

Experiments settings

- Datasets
 - ✓ Five datasets : Flowers, Cars, Sketch, CUB, and WikiArt
- Methods of Comparison
 - ✓ Maximum performance bound is fine-tuning
 - ✓ Minimum performance bound is fixed feature extractor, which shares all layers
 - ✓ Compare with Piggyback, SpotTune, PackNet, and Residual Adapters
- Metrics
 - ✓ Top-1 accuracy on each task
- Parameters
 - ✓ Sweep over $\lambda \in \{0.25, 0.5, 0.75\}$
 - ✓ Fix the threshold $\tau = 0.1$ for all datasets

Experiment Results (Incremental MTL)

- Results on ImageNet-to-Sketch
 - ✓ TAPS outperforms other methods on several datasets
 - ✓ For Car dataset, TAPS does not outperform other methods
 - ✓ Best results are obtained with $\lambda=0.0$, suggesting that most layers needs to be adapted for optimal performance

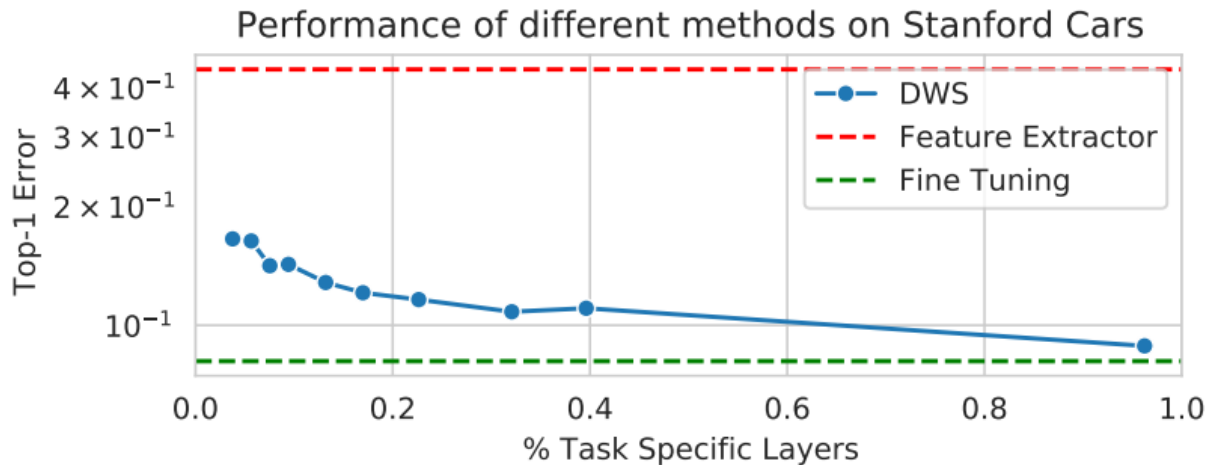


Figure 2. Accuracy vs Task Specific Layers

	Param Count	Flowers	WikiArt	Sketch	Cars	CUB
Fine-Tuning	6×	95.73	78.02	81.83	91.89	83.61
Feature Extractor	1×	89.14	61.74	65.90	55.52	63.46
Piggyback [23]	6×	(2.25×	94.76	71.33	79.91	89.62
Packnet [24] →	(1.60×	93.00	69.40	76.20	86.10	80.40
Packnet [24] ←	(1.60×	90.60	70.3	78.7	80.0	71.4
Spot-tune [9]	7×	(7×	96.34	75.77	80.2	92.4
WTPB [25]	6×	(2.25×	96.50	74.8	80.2	91.5
BA ² [2]	3.8×	(1.71×	95.74	72.32	79.28	92.14
TAPS	4.12×		96.68	76.94	80.74	89.76

Table 1. Performance of various method using a ResNet-50 model on ImageNet-to-Sketch benchmark

Experiment Results (Incremental MTL)

- Task-specific layers
 - ✓ Final convolutional layers are always adapted
 - ✓ Interestingly, some of the middle layers are also always active (layer 26)
 - ✓ For Sketch task, first convolutional layer is adapted
 - ✓ Sketch task has **differing low-level features** compared to ImageNet

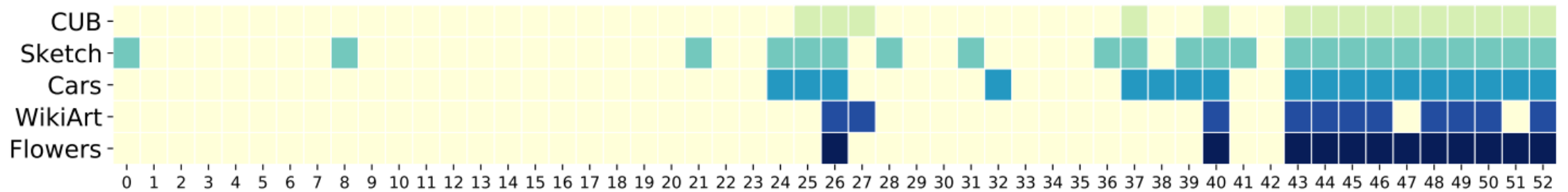


Figure 3. Task-specific layers for different datasets

Experiment Results (Incremental MTL)

- Effect of choice of pre-trained model
 - ✓ Replace the base ImageNet model with a Place-365 model and applied TAPS
 - ✓ The number of task-specific layers increases for every task (25.91% -> 36.60%)
 - ✓ Place-365 may not be well suited for object classification

	Flowers	WikiArt	Sketch	Cars	CUB
Resnet - 50					
% Addl. Parameters					
Imagenet	65.50	52.82	75.87	41.85	70.61
Places	75.08	68.72	74.59	75.02	74.72
% Task Specific Layers					
Imagenet	22.64	20.75	43.40	14.47	28.30
Places	35.85	28.30	50.94	33.96	33.96

Table 7. Effect of Pretrained Model on Task specific parameters

Experiment Results (Incremental MTL)

- Effect of Architecture Choice
 - ✓ Evaluate TAPS on DenseNet, compare with Piggyback
 - ✓ The number of task-specific layers are high in DenseNet-121 compared to ResNet-50
 - ✓ Might be because of the extra skip connections
 - ✓ For ViT (ViT-S/16), fewer parameters are mad task-specific compared to CNNs

	Flowers	WikiArt	Sketch	Cars	CUB
Percentage of Additional Parameters					
DenseNet-121	80.2	41.2	58.5	50.4	43.8
ViT-S/16	41.3	30.4	24.1	26.1	41.3
ResNet-50	65.5	52.8	75.9	41.9	70.6
Percentage of Task Specific Layers					
DenseNet-121	69.4	22.5	41.1	28.3	23.9
ViT-S/16	54.2	20.8	22.9	37.5	54.2
ResNet-50	22.6	20.8	43.4	14.5	28.3

Table 2. Percentage of additional parameters and layers

	Flowers	WikiArt	Sketch	Cars	CUB
DenseNet-121					
Fine-Tuning	95.6	77.0	81.1	89.5	82.6
Piggyback	94.7	70.4	79.7	89.1	80.5
TAPS	95.8	73.6	80.2	88.0	80.9
ViT-S/16					
Fine-Tuning	99.3	82.6	81.9	89.2	88.9
TAPS	99.1	82.3	82.2	88.7	88.4

Table 3. Accuracy of various methods across architectures and datasets

Experiment Results (Incremental MTL)

- Transformers
 - ✓ Very different pattern from those of CNNs
 - ✓ In case of CNN, lower layers tend to be task agnostic and final layers task-specific
 - ✓ Layers throughout the whole network tend to be adapted to be the task
 - ✓ Attention and projection layers tend to be adapted, whereas MLP layers are fixed

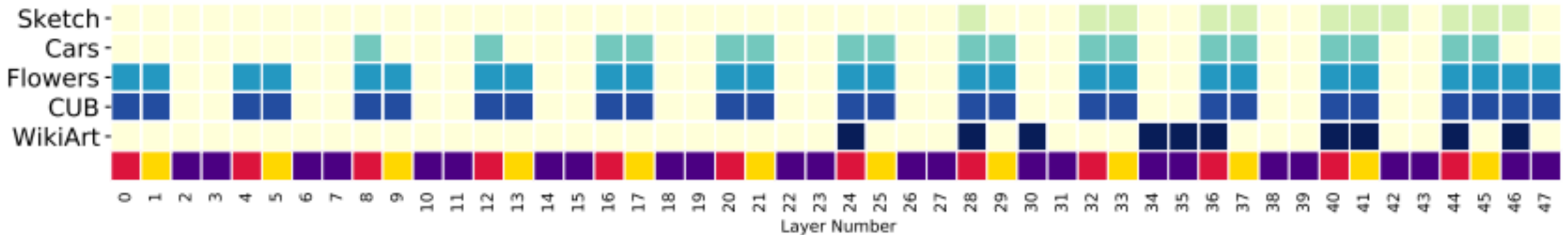
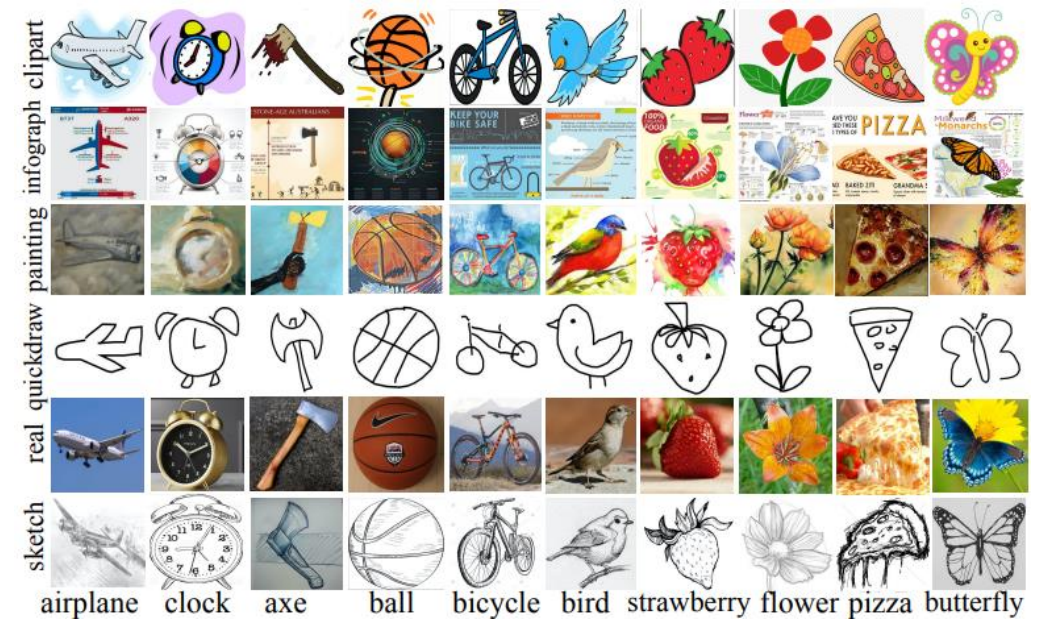


Figure 4. Task-specific layers for different datasets for the ViT model

Experiment Results (Joint MTL)

- Setting
 - ✓ Compare TAPS with AdaShare
 - ✓ Multiple tasks are learned together with shared backbones and independent task heads
 - ✓ Compare performance on the DomainNet datasets with ResNet-34
- DomainNet
 - ✓ Contains the same labels across 6 domains
 - ✓ Task competition can be observed
 - ✓ Originally used for Multi-source domain adaptation



Experiment Results (Joint MTL)

- Select-or-Skip vs Add-or-Not
 - ✓ TAPS outperforms AdaShare in both incremental and joint multi-task setting across all six domain
 - ✓ AdaShare reduces the number of residual blocks of the network (select-or-skip strategy)
 - ✓ TAPS, however, preserves the whole capacity of the base network
- Incremental Training vs Joint Training
 - ✓ Full fine-tuning for each task (Incremental, Fine-tuning) yields the best performance (competition exists)
 - ✓ TABS show relatively stable performance

MTL Setting	Method	Params	Real	Painting	Quickdraw	Clipart	Infograph	Sketch	Mean
Joint	Fine-tuning	1×	75.01	66.13	54.72	75.00	36.35	65.55	62.12
	AdaShare	1×	76.90	67.90	61.17	75.88	31.52	63.96	62.88
	TAPS	1.46×	78.91	67.91	70.18	76.98	39.30	67.81	66.84
Incremental	Fine-tuning	6×	<u>81.51</u>	<u>69.90</u>	<u>73.17</u>	74.08	<u>40.38</u>	<u>67.39</u>	<u>67.73</u>
	AdaShare	5.73×	79.39	65.74	68.15	74.45	34.11	64.15	64.33
	AdaShare ResNet-50	4.99×	78.71	64.01	67.00	73.07	31.19	63.40	62.90
	TAPS	4.90×	80.28	67.28	71.79	74.85	38.21	66.66	66.51

Table 5. TAPS vs AdaShare

4. Conclusion

Summary

- Contributions
 - 1) Propose TAPS, a method for differentially learning which layers to tune
 - 2) Show that TAPS can optimize for accuracy or efficiency
 - 3) TAPS enables efficient incremental and joint multi-task learning without competition or forgetting
- Limitations
 - 1) Tasks do not share task-specific layers with each other
 - 2) Even though the joint training approach can mitigate the issue, retraining on all tasks become infeasible

Thank you